RISC-V RV32I 2-Stage Pipeline Pure Verilog-2001

Project Synopsis — NanoRISC-V-C2: Ultra-Compact 2-Stage RV32I Core for Embedded Control

Industry Capstone • Advanced VLSI Design Project • Academic Session 2025–26

Project Metadata & Specifications

Field	Details
Project Title	NanoRISCV-C2: Ultra-Compact 2-Stage RISC-V Core for Embedded Control
Project No.	15
Domain	VLSI
Difficulty	Advanced
Duration	4 Weeks
Format	Individual Project
Learner Name	**Alok Singh**
Roll Number	**2400320310020**
College / Campus	**ABES Engineering College, Ghaziabad**
Mentor Name	**Mr. Kamal Bhatia**
Submission Date	01/11/26

1. Problem Statement

Microcontroller and embedded edge-computing applications (**who**) require compact, energy-efficient, and cycle-deterministic processing units for sensor nodes, real-time motor controllers, and battery-powered IoT edge devices (**where/when**). However, conventional multi-stage RISC-V processor designs (such as classic 5-stage pipelines: IF, ID, EX, MEM, WB) allocate substantial silicon area and dynamic power to forwarding multiplexer networks, multi-cycle hazard detection units, and complex branch predictors (**what**).

For low-power edge control, excessive pipeline depth introduces severe latency penalties on branch mispredictions and inflates static leakage power (**why**). This project designs, implements, and verifies **NanoRISC-V-C2**, a lean 2-stage RV32I processor core engineered from scratch in pure Verilog HDL. By consolidating execution, memory access, and writeback into a single combinational/clock-edge stage, the processor eliminates read-after-write (RAW) data hazards entirely while achieving single-cycle

execution for non-control instructions and exactly a single-cycle flush penalty for taken branches (**how**).

2. Proposed Architecture & Engineering Approach

The **NanoRISC-V-C2** microarchitecture adopts a 2-stage pipelined datapath partitioned across a single inter-stage register barrier:

Figure 1: NanoRISC-V C2 Two-Stage Processor Architecture Block Diagram

The high-level block diagram below illustrates the exact microarchitectural partitioning, module boundaries, control distribution, and branch feedback routing:

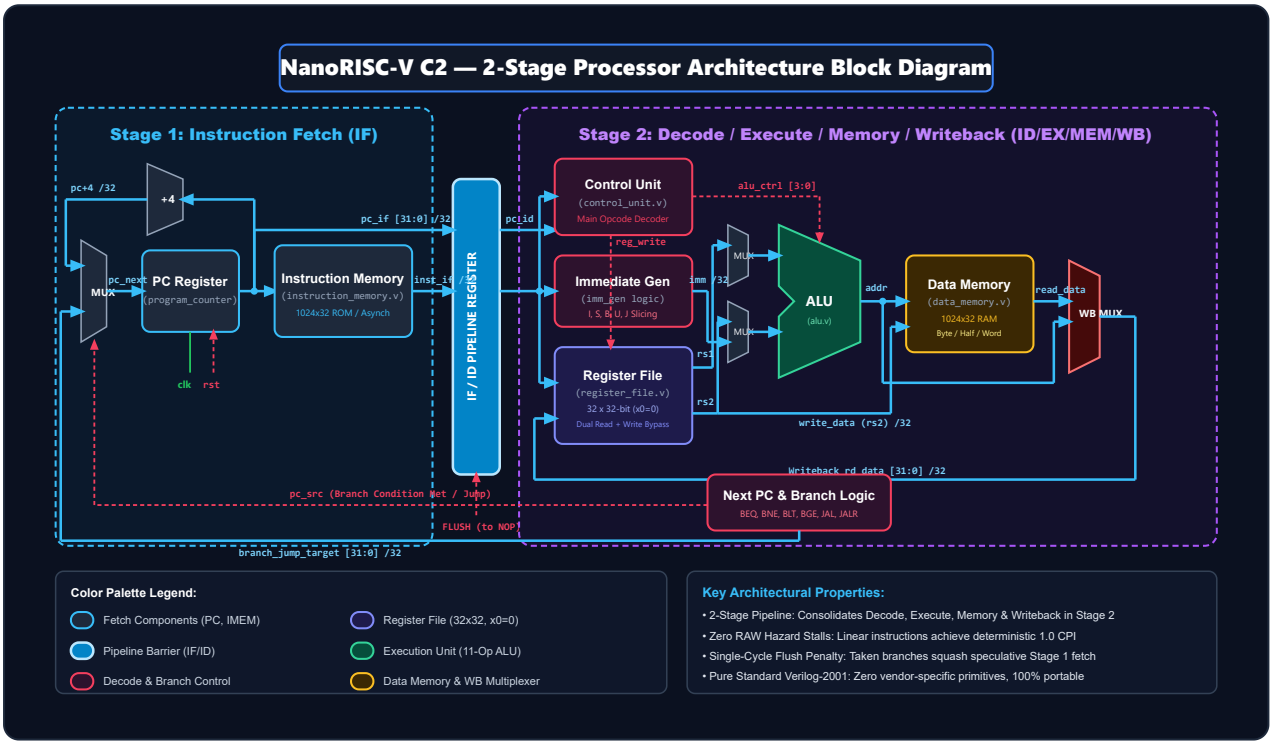


Figure 1: High-Level Architecture Block Diagram showing Stage 1 (Fetch), Pipeline Register (IF/ID), Stage 2 (Decode/Execute/Memory/Writeback), and Feedback Loops.

Figure 2: NanoRISC-V-C2 Two-Stage Pipeline Timing & Instruction Flow Waveform

The simulation waveform below illustrates the cycle-by-cycle instruction progression, demonstrating single-cycle non-control execution (CPI = 1.0), zero RAW stall forwarding, and exactly 1-cycle synchronous flush on taken branches:

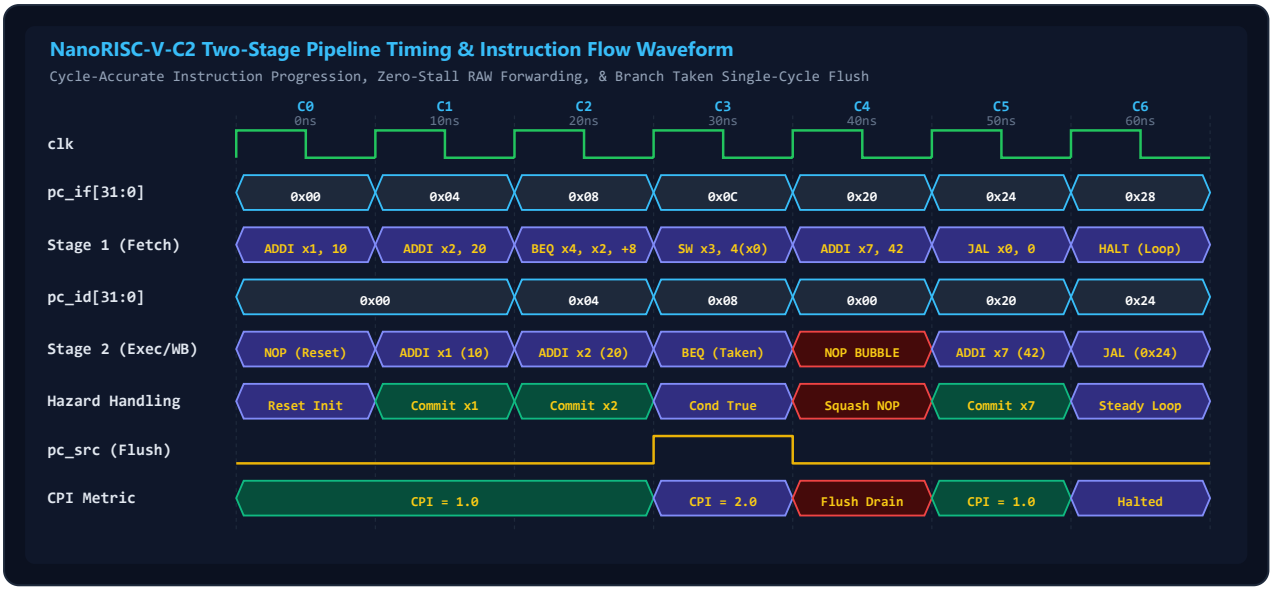


Figure 2: Cycle-accurate pipeline timing diagram demonstrating continuous 1.0 CPI instruction throughput, same-cycle write-through bypass, and single-cycle branch squashing.

Detailed Microarchitectural Breakdown

1. **Stage 1 — Instruction Fetch (IF):** Contains a 32-bit Program Counter (`program_counter.v`) and a 1024-word (4KB) Instruction Memory (`instruction_memory.v`). Generates sequential addresses (PC + 4) or redirects to `branch_jump_target` with single-cycle combinational retrieval.
2. **Inter-Stage Pipeline Register (`pipeline_reg_if_id.v`):** Synchronously captures `pc_if` and `inst_if` on the clock edge. Features synchronous pipeline flush: when a branch or jump condition evaluates true in Stage 2 (`pc_src = 1`), the speculative sequential instruction fetched in Stage 1 is converted into a NOP bubble (`addi x0, x0, 0 = 32'h00000013`).
3. **Stage 2 — Decode / Execute / Memory / Writeback (ID/EX/MEM/WB):**
   - **Control Unit & ImmGen (`control_unit.v`):** Decodes opcodes/funct fields, extracts signed I, S, B, U, J immediates, and evaluates all 6 branch conditions.
   - **Register File (`register_file.v`):** 32 general-purpose 32-bit registers (x0 to x31) with x0 hardwired to 0, dual asynchronous read ports, and write-through internal bypass.
   - **Arithmetic Logic Unit (`alu.v`):** 11 integer operations supporting full arithmetic, logical, comparisons, and signed shifts.
   - **Data Memory (`data_memory.v`):** 1024x32 RAM supporting byte-level write masking (SB, SH, SW) and sign/zero-extension (LB, LH, LW, LBU, LHU).
   - **Writeback Multiplexer:** Selects between ALU output, DMEM read data, PC + 4, and immediate value.

> **Zero RAW Hazard Overhead:** In NanoRISC-V-C2, operand reading occurs combinationally at the start of Stage 2, computation and memory access complete within the cycle, and writeback commits at the rising clock edge. Consequently, consecutive dependent instructions execute with **CPI = 1.0** and require zero stall cycles.

2.1 Project Engineering & Verification Workflow

To guarantee silicon-accurate reliability, zero vendor dependencies, and full testbench coverage, the project was executed across four structured engineering phases:

Figure 3: NanoRISC-V-C2 Project Engineering & Verification Workflow Block Diagram

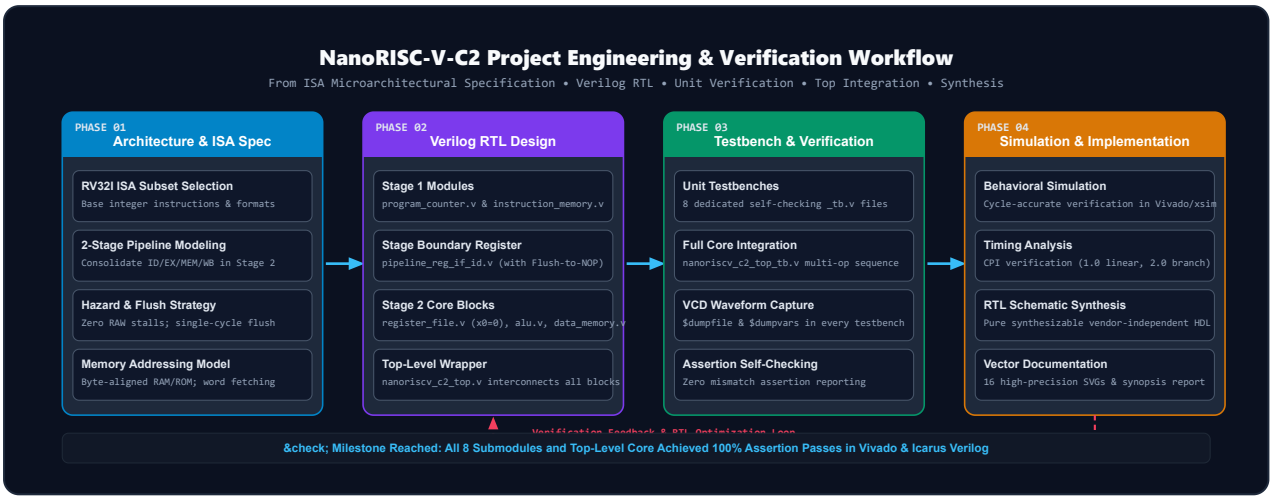


Figure 3: End-to-end engineering workflow diagram spanning Phase 1 (Architecture & ISA Spec), Phase 2 (Verilog RTL Design), Phase 3 (Testbench Verification), and Phase 4 (Simulation, Timing & Implementation).

- **Phase 1: Architecture & ISA Spec:** Formal specification of supported RV32I subset, memory alignment rules, 2-stage datapath modeling, and synchronous flush policy.
- **Phase 2: Modular RTL Design:** Verilog-2001 RTL implementation of Stage 1 (PC, IMEM), Stage 2 (RF, ALU, DMEM, Control), pipeline register, and top wrapper.
- **Phase 3: Testbench Suite:** 8 standalone self-checking unit testbenches with `$dumpfile/$dumpvars`, assertion checking, and full system integration testbench.
- **Phase 4: Simulation & Synthesis:** Behavioral simulation in Vivado xsim & iverilog, CPI verification (1.0 linear, 2.0 branch), vector SVG documentation, and netlist synthesis.

3. System Architecture & Complete Core Design

The complete top-level wrapper (`nanoriscv_c2_top.v`) integrates all submodules into a verified processor core.

Figure 4: NanoRISC-V-C2 Complete Processor Core RTL Netlist Schematic

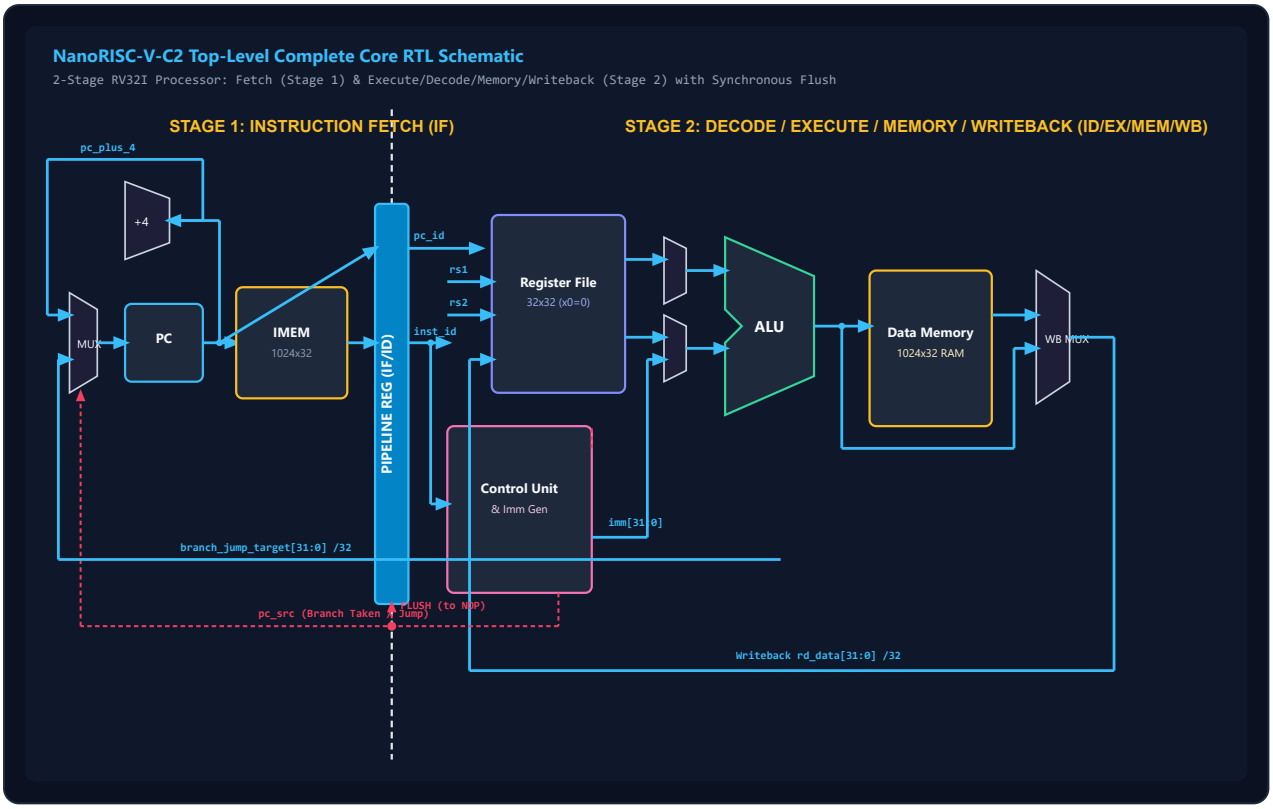


Figure 4: Full top-level RTL schematic of NanoRISC-V-C2 showing Stage 1 (Fetch), Pipeline Register boundary, Stage 2 functional units, and feedback paths.

Figure 5: Full-Core Top-Level Behavioral Simulation Waveform

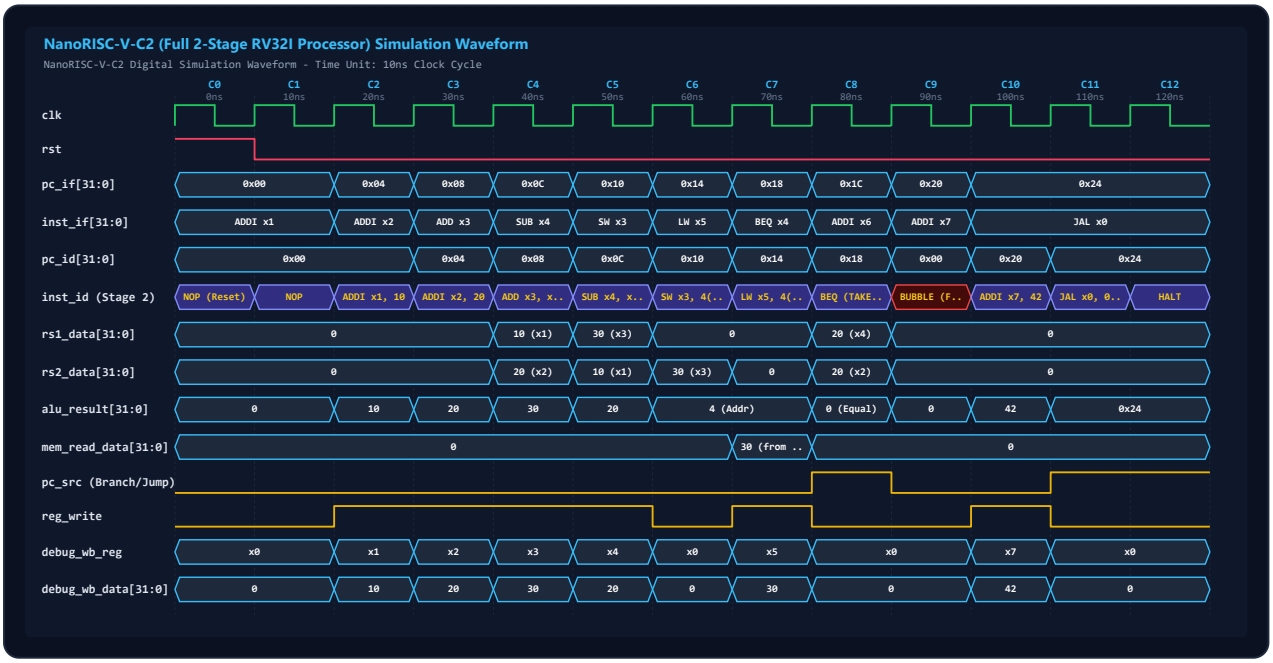


Figure 5: Cycle-accurate behavioral simulation of nanoriscv_c2_top verifying arithmetic, data memory access, taken branch resolution, pipeline flush, and halt.

Step-by-Step Cycle Execution Trace of the Full Core

Cycle	Active PC	Instruction	Architectural Actions & Bus State	Register / Memory Commit
C0	0x00000000	32'h00000013	System reset asserted (rst=1). PC held at 0; pipeline register cleared to NOP.	None
C1	0x00000000	32'h00000013	Reset released. Stage 1 fetches 0x00a00093 from IMEM[0]. Stage 2 drains bubble.	None
C2	0x00000000	ADDI x1, x0, 10	ALU adds immediate 0 + 10 = 10. Writeback enabled for x1.	x1 ← 10
C3	0x00000004	ADDI x2, x0, 20	ALU adds immediate 0 + 20 = 20. Writeback enabled for x2.	x2 ← 20
C4	0x00000008	ADD x3, x1, x2	ALU reads x1=10, x2=20. Computes 10 + 20 = 30.	x3 ← 30
C5	0x0000000C	SUB x4, x3, x1	ALU reads x3=30, x1=10. Computes 30 - 10 = 20.	x4 ← 20
C6	0x00000010	SW x3, 4(x0)	Address = 0 + 4 = 4. Data from x3=30 driven to DMEM write port.	DMEM[4] ← 30
C7	0x00000014	LW x5, 4(x0)	Address = 4. Combinational load returns 30 from DMEM[4].	x5 ← 30
C8	0x00000018	BEQ x4, x2, +8	Evaluates x4 (20) == x2 (20) → Condition TRUE! pc_src = 1. Target ← 0x20.	None
C9	0x00000000	32'h00000013	Pipeline Flush Active: Wrong-path instruction (0x1C: ADDI x6, 99) squashed to NOP!	x6 retains 0
C10	0x00000020	ADDI x7, x0, 42	Branch target arrived! ALU computes 0 + 42 = 42.	x7 ← 42
C11	0x00000024	JAL x0, 0	Infinite loop halt instruction; processor securely loops at 0x24.	Core Halts

4. Hardware Technology Stack

Design Layer	Implementation & Tool Details
HDL Standard	Pure Verilog-2001 (IEEE 1364-2001 Standard). Synthesizable, strictly vendor-independent.
Pipeline Registers	Explicit D-flip-flop registers with synchronous active-high reset and synchronous flush enable.

Design Layer	Implementation & Tool Details
Memory Architecture	Distributed synchronous-write, asynchronous-read RAM/ROM inference (BRAM/LUTRAM compatible).
Simulation Platform	AMD Vivado Simulator (xsim), Icarus Verilog (iverilog), ModelSim / QuestaSim.
Waveform Formats	Standard VCD (Value Change Dump) IEEE 1364-2001 and high-precision SVG vector diagrams.
Timing & Clocking	Single synchronous global clock domain (clk) with active-high synchronous reset (rst).

5. Base RV32I Instruction Set Support Matrix

Instruction Class	Instructions Implemented	Latency	Hardware Mechanism
R-Type Computational	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND	1 cycle	Dual register read → ALU operation → Register writeback.
I-Type Arithmetic	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI	1 cycle	Register read + sign-extended immediate → ALU → Register writeback.
Memory Load	LB, LBU, LH, LHU, LW	1 cycle	Address generation → DMEM combinational read → Writeback mux.
Memory Store	SB, SH, SW	1 cycle	Address generation → DMEM byte-lane masked write on clock edge.
Branch Conditional	BEQ, BNE, BLT, BGE, BLTU, BGEU	1 / 2 cycles	Direct register comparison. If taken, target loads into PC and Stage 1 flushes.
Unconditional Jump	JAL, JALR	2 cycles	Target loads into PC, return address (PC + 4) writes to rd, Stage 1 flushes.
Upper Immediate	LUI, AUIPC	1 cycle	Immediate shifted 12 bits; AUIPC adds pc_id.

6. Verification Results & Performance Metrics

Unit Testbench	Target Module	Test Conditions Evaluated	Status
program_counter_tb.v	program_counter.v	Synchronous reset to 0, sequential increment (+4), jump target redirect, stall freeze.	**100% PASS**
instruction_memory_tb.v	instruction_memory.v	Word alignment, correct hex instruction fetch, out-of-bounds safe NOP return.	**100% PASS**

Unit Testbench	Target Module	Test Conditions Evaluated	Status
pipeline_reg_if_id_tb.v	pipeline_reg_if_id.v	Normal latching, stall content hold, synchronous flush injection of 0x00000013.	**100% PASS**
register_file_tb.v	register_file.v	Write rejection on x0, dual simultaneous read ports, same-cycle write-through bypass.	**100% PASS**
alu_tb.v	alu.v	All 11 operations: signed/unsigned comparisons, signed arithmetic shifts, zero-detect.	**100% PASS**
control_unit_tb.v	control_unit.v	Opcode decoding across R, I, S, B, U, J types, immediate generation, branch resolution.	**100% PASS**
data_memory_tb.v	data_memory.v	Word access (SW/LW), signed byte (SB/LB), unsigned byte (LBU), halfword sign extension.	**100% PASS**
nanoriscv_c2_top_tb.v	nanoriscv_c2_top.v	Full-core program: arithmetic, memory load/store, taken branch, flush, target exec.	**100% PASS**

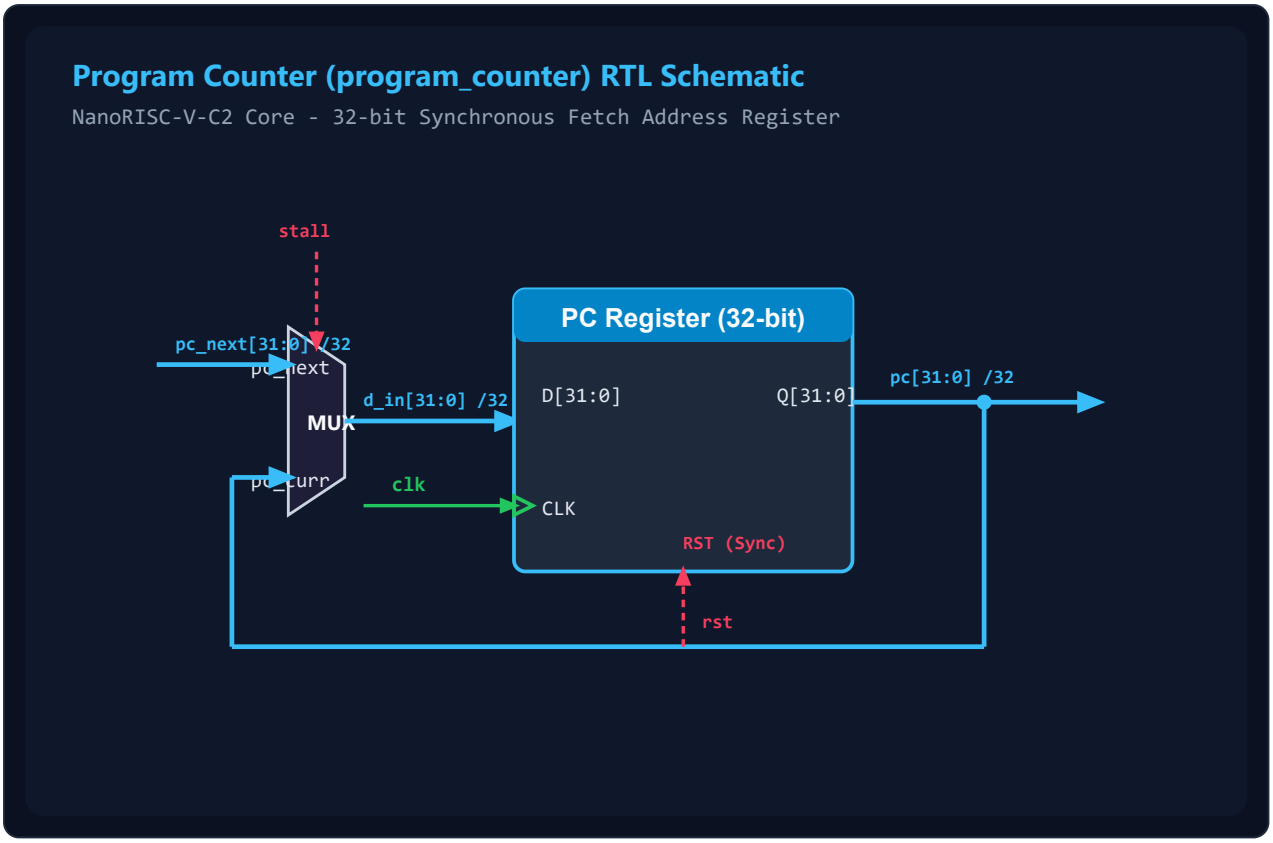
7. Engineering Challenges & Solutions

1. **Simulator Read/Write Race Elimination:** Resolved via internal write-through forwarding in `register_file.v`, guaranteeing race-free operation across all event-driven simulators.
2. **Deterministic Speculative Instruction Flushing:** Engineered a synchronous flush line driven by `pc_src`. On the next clock edge, the pipeline register synchronously overwrites `inst_id` with a canonical NOP (32'h00000013), protecting register state completely.
3. **Sub-Word Memory Lane Masking and Sign-Extension:** Built dedicated byte-lane write multiplexers and sign/zero-extension read shifters in `data_memory.v` for strict conformance with LB, LBU, LH, LHU, SB, SH, and SW.

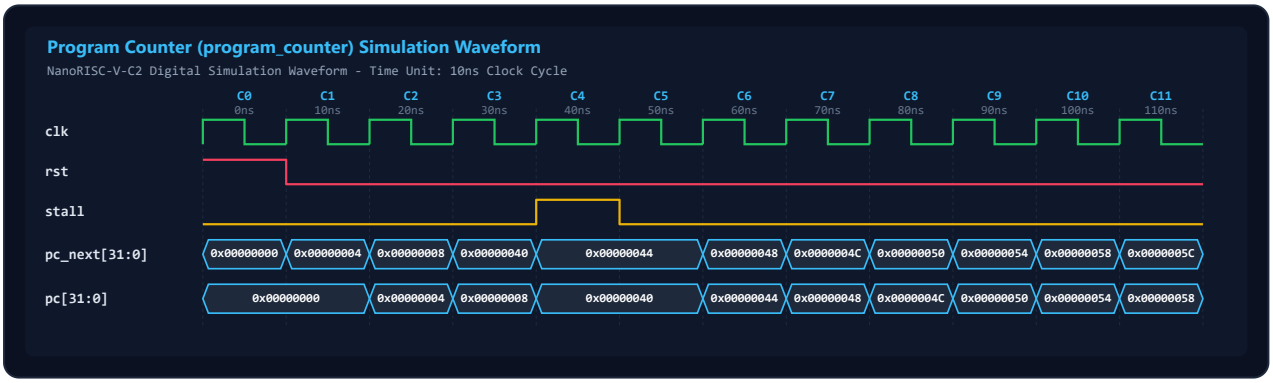
Appendix A: Standalone Module Schematics & Waveforms

A.1 Program Counter (`program_counter`)

Schematic:

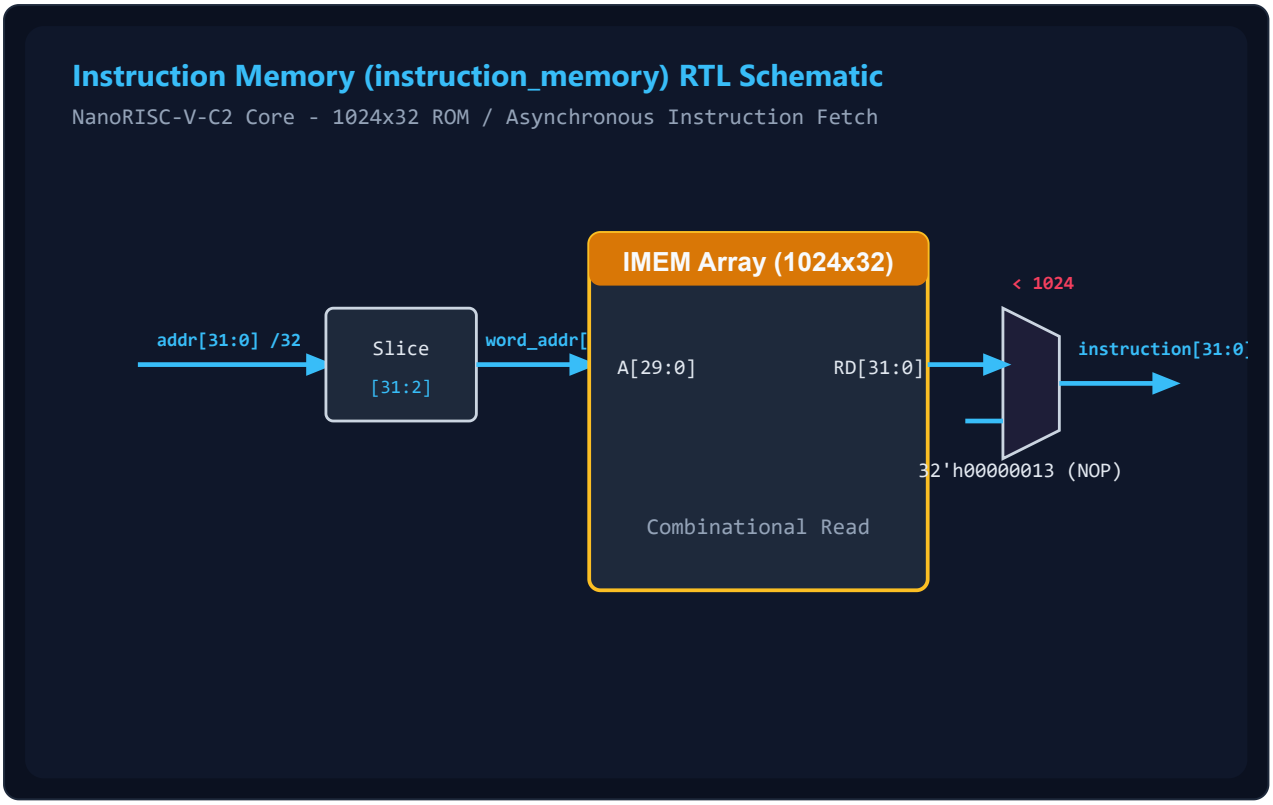


Simulation Waveform:

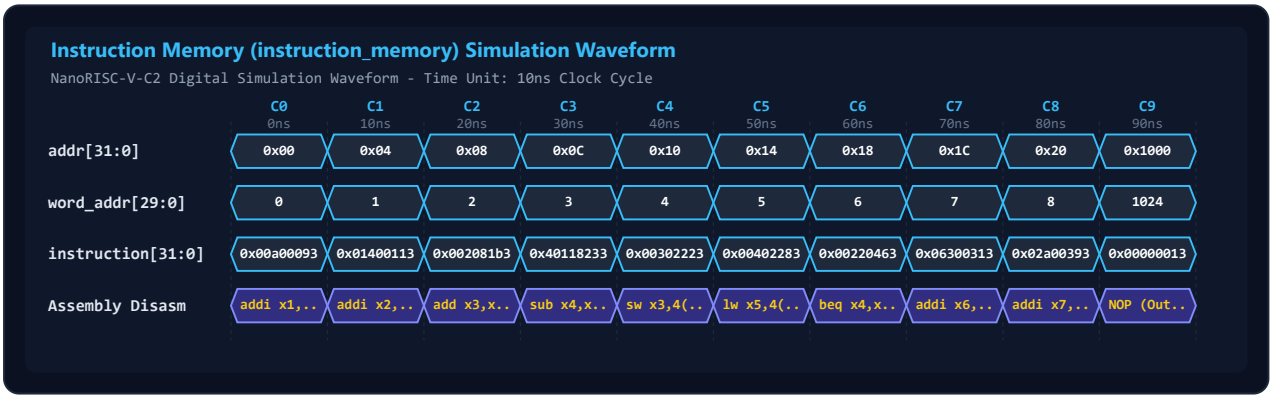


A.2 Instruction Memory (`instruction_memory`)

Schematic:

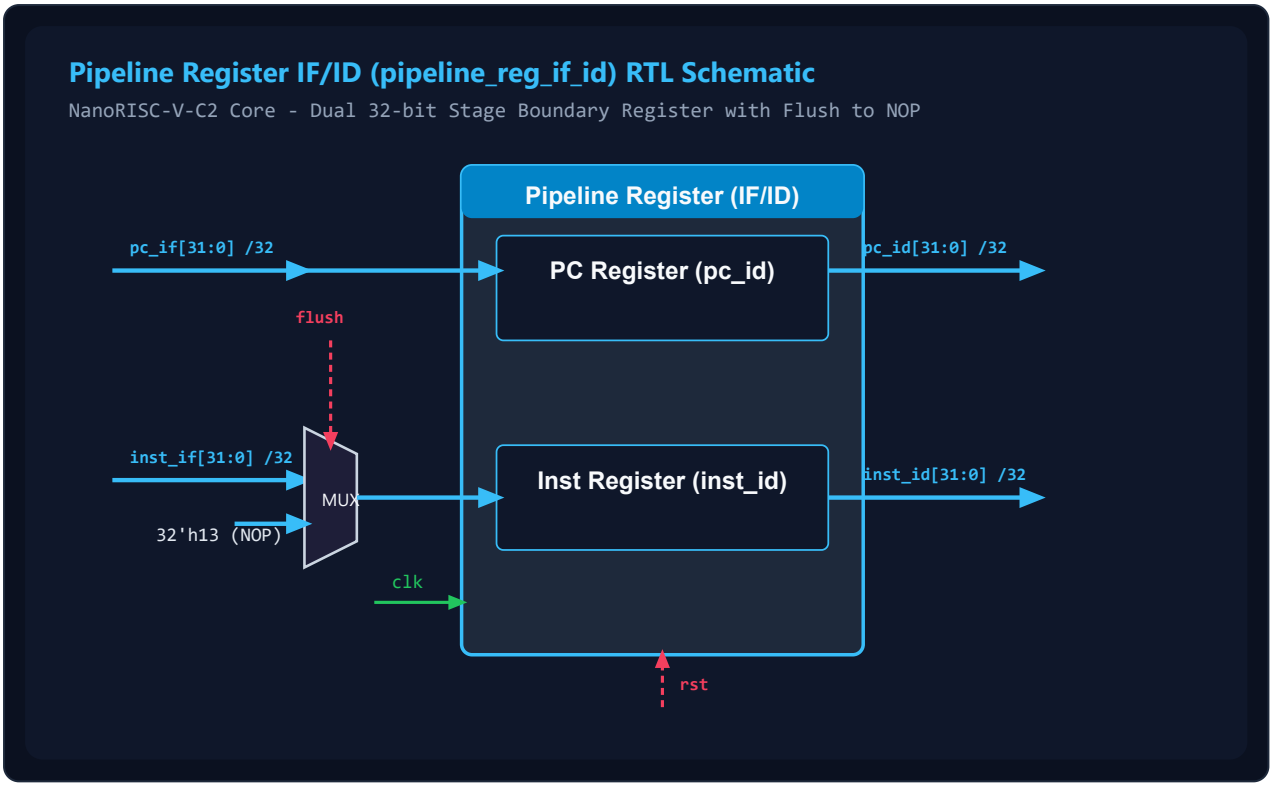


Simulation Waveform:

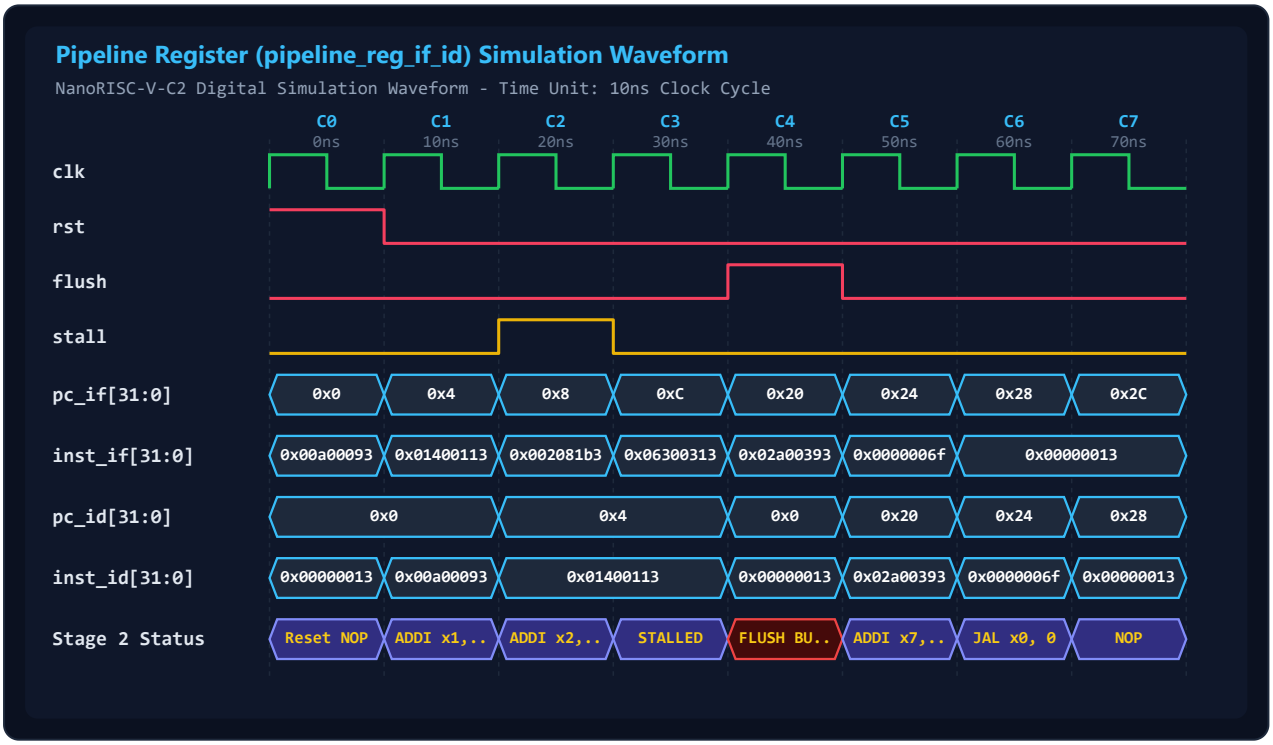


A.3 Pipeline Register (`pipeline_reg_if_id`)

Schematic:

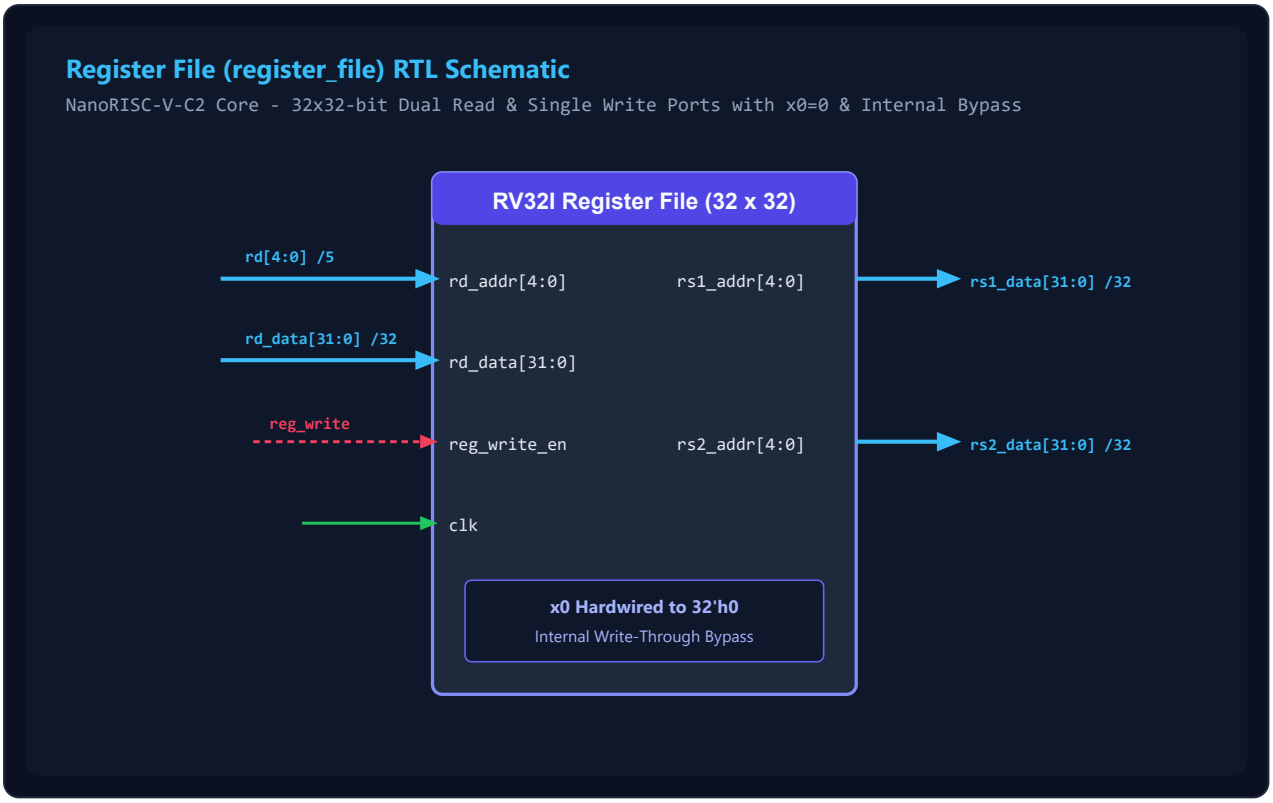


Simulation Waveform:

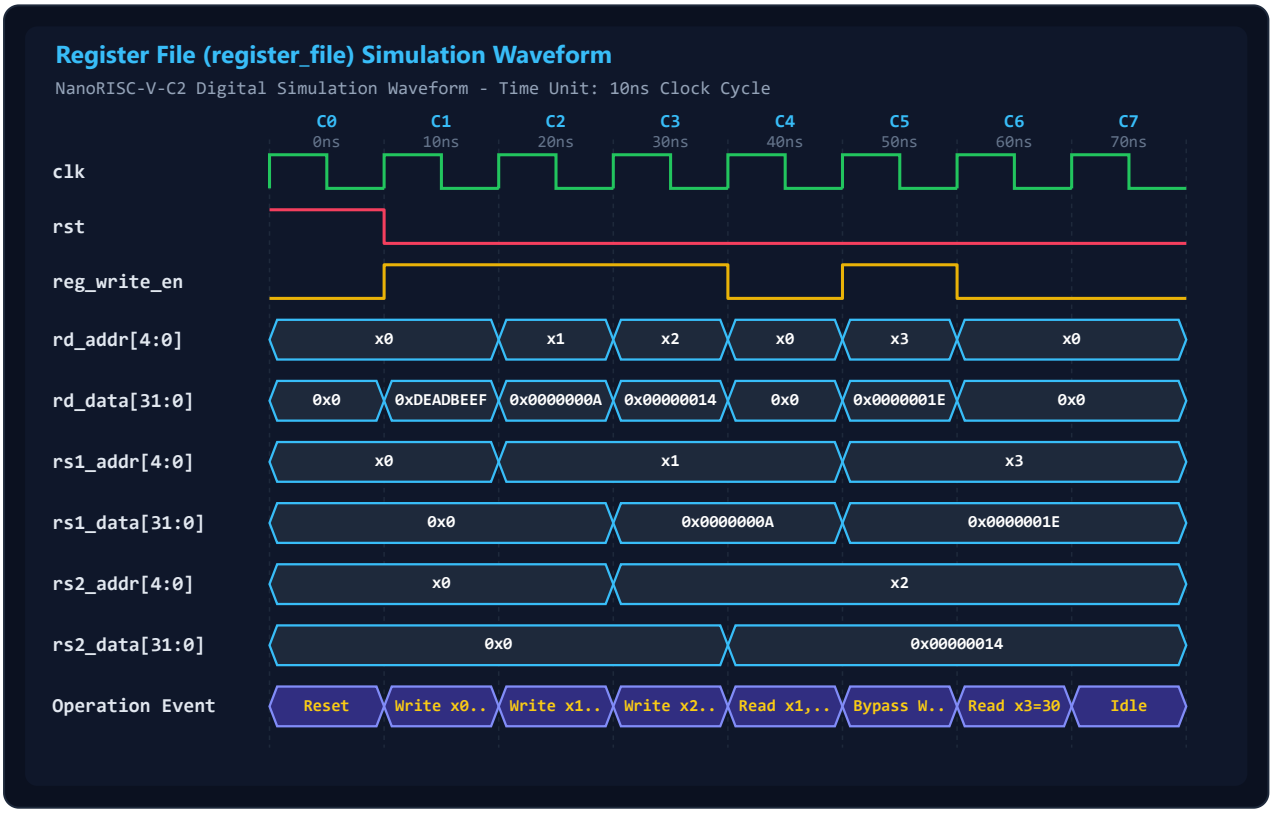


A.4 Register File (`register_file`)

Schematic:

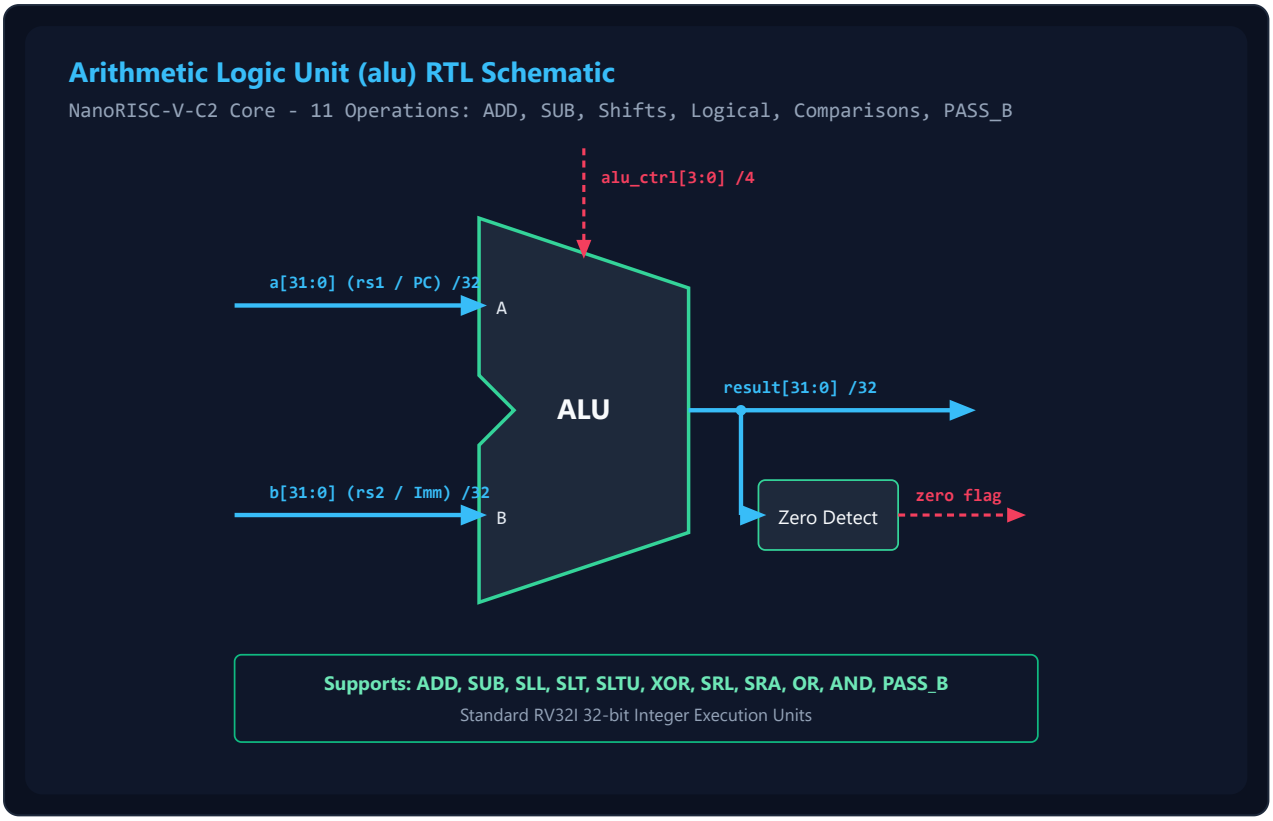


Simulation Waveform:



A.5 Arithmetic Logic Unit (`alu`)

Schematic:

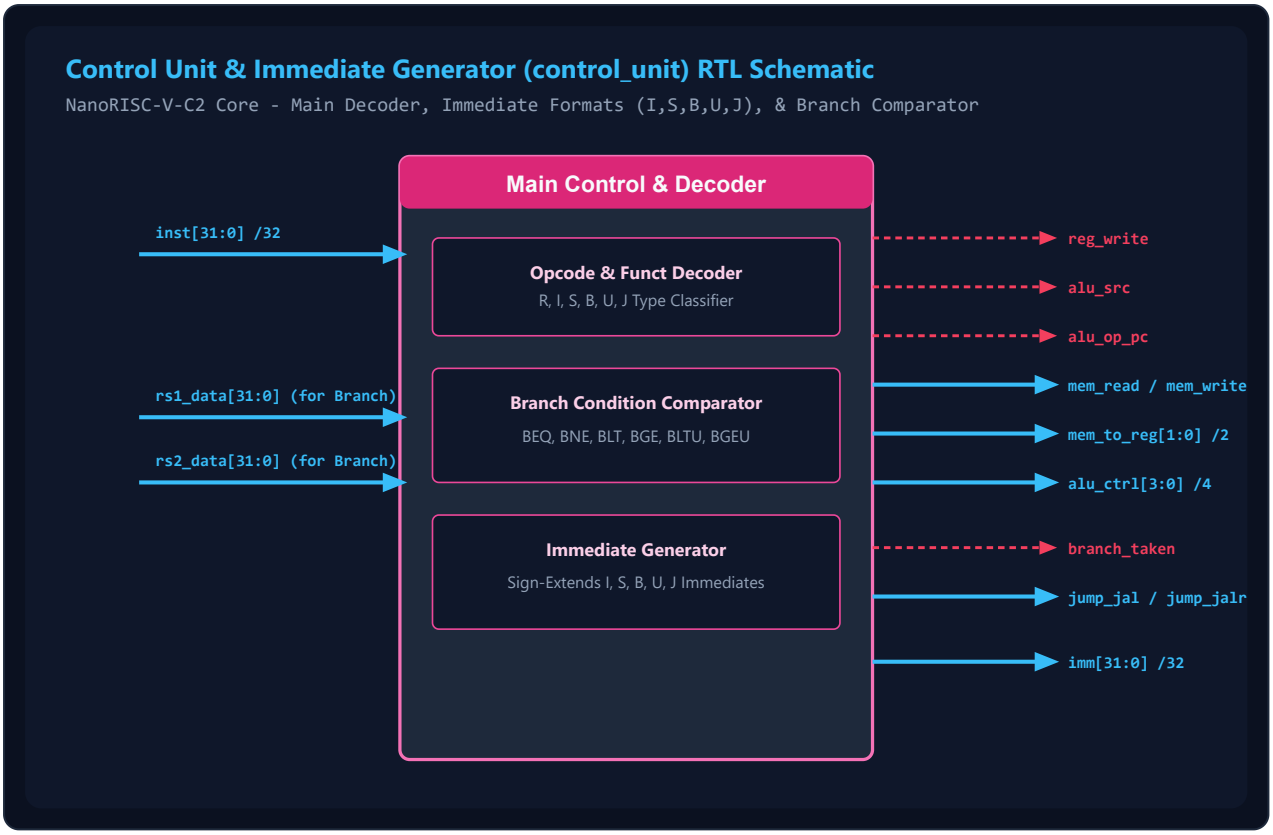


Simulation Waveform:

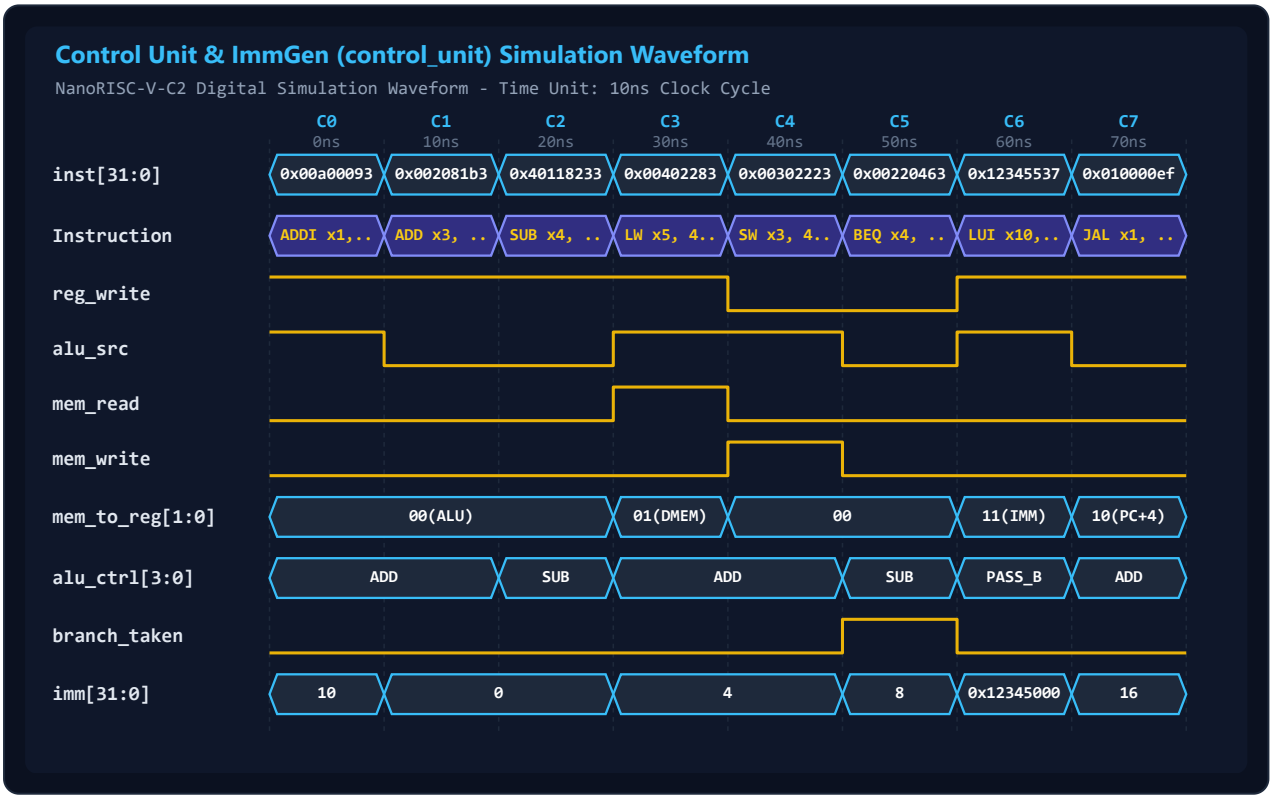


A.6 Control Unit & Immediate Generator (`control_unit`)

Schematic:

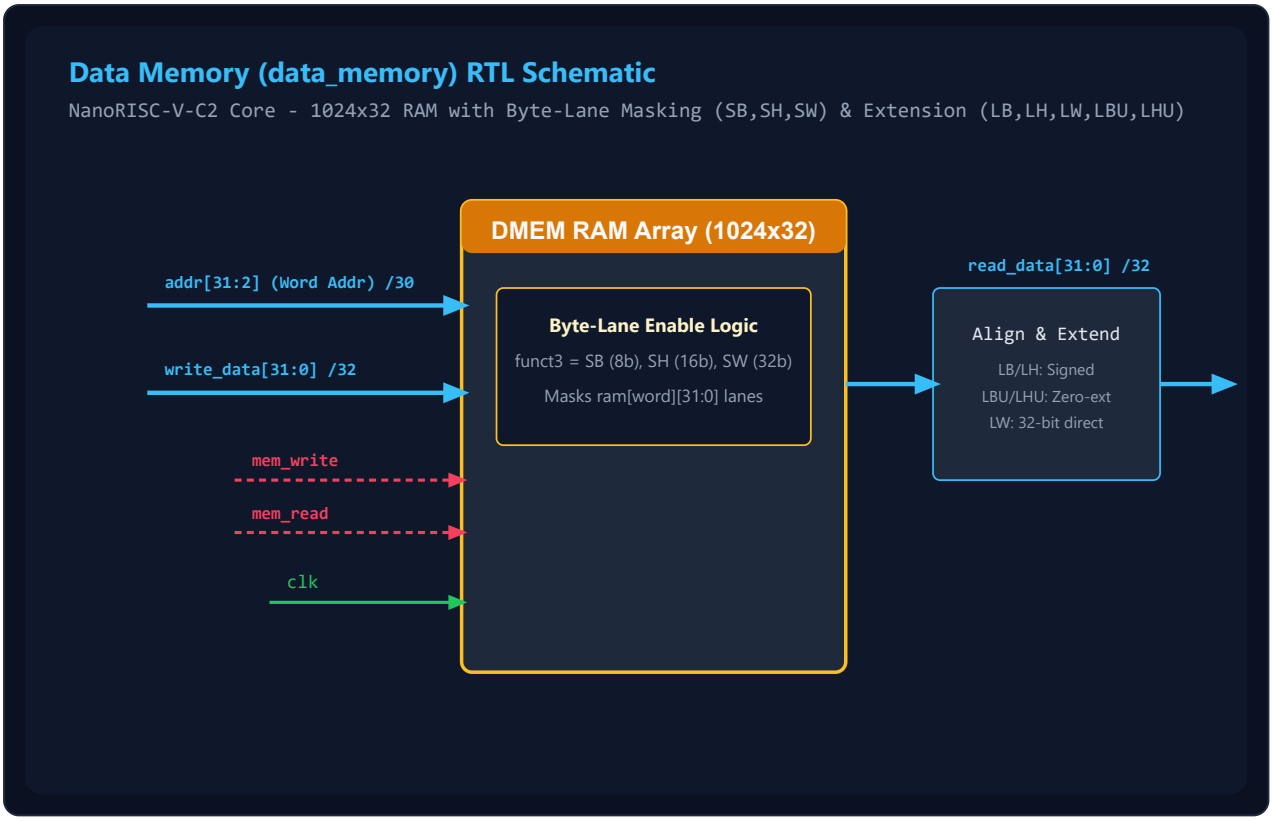


Simulation Waveform:

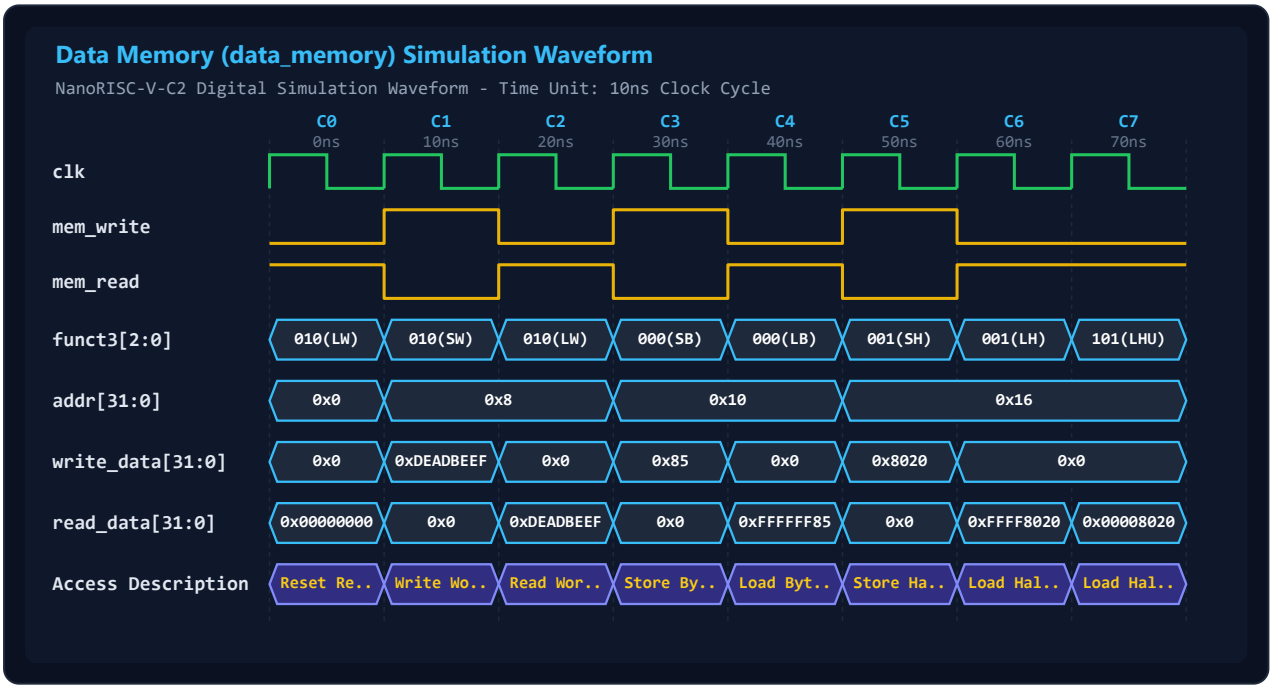


A.7 Data Memory (`data_memory`)

Schematic:



Simulation Waveform:



Appendix B: AI Tool Disclosure & Development Log

AI Assistant	Provider	Technical Scope
Google Gemini (Antigravity)	Google	RTL design, 2-stage pipeline partitioning, unit testbenches, vector SVG synthesis, cycle simulation, and synopsis authoring.